

Lecture 11: Word embeddings

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will

1. Understand Word2Vec (CBOW and Skip-gram)
2. Explain how negative sampling makes training efficient
3. Compare Word2Vec, GloVe, and FastText
4. Use word embeddings to solve analogies and compute similarities
5. Recognize biases in word embeddings

The Word2Vec revolution

Word2Vec showed that neural networks can learn powerful semantic representations from raw text alone.

From count-based to prediction-based embeddings

Count-based methods (LSA, LDA)

- Input: term-document matrices
- Uses term co-occurrence statistics across documents
- Captures **global** patterns and linear relationships

Prediction-based methods (Word2Vec)

- Input: raw text sequences
- Learn to predict context from word (and vice versa)
- Use **local** context to learn embeddings
- Captures non-linear relationships

Core difference

Beyond the implementation details, there is an even more fundamental difference between the two classes of approaches. Whereas count-based methods learn by counting (where embeddings are the end goal), prediction-based methods like Word2Vec learn by predicting; the embeddings come "for free" as a consequence of learning to predict.

Word2Vec: words that appear in similar contexts should have similar embeddings

Context

In NLP, **context** refers to the words surrounding a target word within a specified window size (i.e., number of preceding and proceeding words/tokens).

Example context window

"The quick brown **[TARGET]** jumped over the lazy dog"

Context: [The, quick, brown, jumped, over, the, lazy, dog]

Target: fox

Key idea

- Train a model to predict target from context (or vice versa)
- The learned **weights** become our embedding vectors!
- We don't actually care about the prediction task; we just want the embeddings!

CBOW: Continuous Bag of Words

CBOW architecture

Given context words, predict the center word

Example

Input: "the, cat, on, the" → **Predict:** "sat"

Characteristics

- Quick training (averages context vectors)
- Better performance for frequent words (word appears in many contexts)
- Smooths over distributional information

Skip-gram

Skip-gram architecture

Given center word, predict the context words

Example

Input: "sat" → **Predict:** "the, cat, on, the"

Characteristics

- Slower than CBOW but often results in better quality embeddings
- Better performance for rare words (more focus on specific contexts where the word appears)
- **Most commonly used** in practice

Context windows: worked example

Sentence: "The cat sat on the mat" | **Window size = 2**

1	Position 0: "The" → Context: [cat, sat]
2	Position 1: "cat" → Context: [The, sat, on]
3	Position 2: "sat" → Context: [The, cat, on, the]
4	Position 3: "on" → Context: [cat, sat, the, mat]
5	Position 4: "the" → Context: [sat, on, mat]
6	Position 5: "mat" → Context: [on, the]

Skip-gram training pairs (target → context):

1	(The, cat), (The, sat), (cat, The), (cat, sat), (cat, on), ...
---	--

The efficiency problem

Challenge

Softmax over entire vocabulary is expensive!

$$P(w_{context} | w_{center}) = \frac{\exp(v_{context}^T v_{center})}{\sum_{w=1}^V \exp(v_w^T v_{center})}$$

For 100k vocabulary: 100k exponentials per training example!

Solution: Negative sampling

Key insight

Instead of predicting across all words, create binary classification:

- **Positive:** Actual context word (label = 1)
- **Negative:** k random words (label = 0)

Efficiency gain

Training becomes $O(k)$ instead of $O(V)$!

Typical k : 5–20 negative samples per positive example.

Negative sampling example

Training pair: ("cat", "sat") — cat is center, sat is context

```
1  # Positive sample: Does "sat" appear near "cat"? YES (label=1)
2  positive_pair = ("cat", "sat", label=1)
3
4  # Negative samples: Random words that DON'T appear near "cat"
5  negative_pairs = [
6      ("cat", "algorithm", label=0),
7      ("cat", "president", label=0),
8      ("cat", "quantum", label=0),
9  ]
10
11 # Train binary classifier: Is this a real context pair?
12 # Instead of 100k-way softmax → just 4 binary predictions!
```

The famous result: vector arithmetic

$$\overrightarrow{\text{King}} - \overrightarrow{\text{Man}} + \overrightarrow{\text{Woman}} \approx \overrightarrow{\text{Queen}}$$

Other analogies

- $\overrightarrow{\text{Paris}} - \overrightarrow{\text{France}} + \overrightarrow{\text{Italy}} \approx \overrightarrow{\text{Rome}}$
- $\overrightarrow{\text{Walking}} - \overrightarrow{\text{Walk}} + \overrightarrow{\text{Swim}} \approx \overrightarrow{\text{Swimming}}$
- $\overrightarrow{\text{Bigger}} - \overrightarrow{\text{Big}} + \overrightarrow{\text{Small}} \approx \overrightarrow{\text{Smaller}}$

Why does this work?

Subtracting "man" removes the "male" component; adding "woman" adds the "female" component. Result: "female royal" = queen!

Deeper insight

By learning embeddings that predict context, Word2Vec ends up capturing how each word relates to the contexts in which it appears. It's like each word has some "pull" on the surrounding words (and vice versa). When we do vector arithmetic, we're manipulating these "pulls" to find words that fit the same relational pattern.

Word2Vec in Python

```
1  from gensim.models import Word2Vec
2  import gensim.downloader as api
3
4  # Load pre-trained model (3B words, 300 dimensions)
5  model = api.load('word2vec-google-news-300')
6
7  # Find similar words
8  model.most_similar('computer', topn=5)
9  # [('computers', 0.72), ('laptop', 0.69), ('PC', 0.68), ...]
10
11 # Word analogies: king - man + woman = ?
12 model.most_similar(positive=['king', 'woman'], negative=['man'], topn=1)
13 # [('queen', 0.71)]
14
15 # Train your own
16 sentences = [['the', 'cat', 'sat'], ['the', 'dog', 'ran']]
17 custom_model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, sg=1)
```

GloVe: Global Vectors

Further reading

Pennington, Socher, and Manning (2014, *EMNLP*) GloVe: Global Vectors for Word Representation.

Key insight

Ratios of co-occurrence probabilities encode meaning better than raw probabilities!

How does GloVe work?

Example: how ratios reveal meaning

Probe word	P(w ice)	P(w steam)	Ratio
solid	high	low	>> 1
gas	low	high	<< 1
water	high	high	~ 1

What GloVe does

GloVe learns word vectors whose dot product relates to co-occurrence probability:

$$\vec{w}_i^T \vec{w}_j \approx \log P(i, j)$$

FastText: leveraging *subword information*

Further reading

Bojanowski, Grave, Joulin, and Mikolov (2017, *TACL*) Enriching word vectors with subword information.

Subword information

Subword information refers to the smaller components that make up words, such as character n-grams, prefixes, suffixes, and morphemes. These components can provide valuable insights into the meaning and structure of words, especially for rare or out-of-vocabulary terms.

The Out-of-Vocabulary (OOV) Problem

Word2Vec and GloVe give ONE vector per word:

- New word? Unknown!
- Misspelling? Unknown!
- Rare morphological form? Unknown!

But "running", "runner", "runnable" share morphology, and we can exploit that!

Remember...

Remember back to when we introduced tokens made up of subword units (like Byte Pair Encoding)? FastText takes advantage of this idea by breaking words down into smaller parts, allowing it to create embeddings for words it hasn't seen before.

FastText: character n-grams

Character n-grams

Character n-grams are contiguous sequences of 'n' characters extracted from a word.

Example: character n-grams for 'where'

Character trigrams: `<wh, whe, her, ere, re>`

Word vector = average of n-gram vectors:

$$\vec{w}_{where} = \frac{1}{6}(\vec{z}_{<wh} + \vec{z}_{whe} + \vec{z}_{her} + \vec{z}_{ere} + \vec{z}_{re} + \vec{z}_{<where>})$$

Why is this useful?

- **Handles OOV words!** Can generate vectors for words never seen in training
- **Captures morphology:** "unhappiness" shares n-grams with "unhappy", "happiness"
- **Great for morphologically rich languages:** German, Turkish, Finnish
- **Robust to typos:** misspellings share many n-grams with correct spellings

Bias in word embeddings

Critical issue

Embeddings learn biases from training data!

Gender bias examples

- $\vec{\text{Man}} : \vec{\text{Computer programmer}} :: \vec{\text{Woman}} : \vec{\text{Homemaker}}$
- $\vec{\text{Man}} : \vec{\text{Doctor}} :: \vec{\text{Woman}} : \vec{\text{Nurse}}$

Estimating bias

Example: estimating term-specific gender bias

```
1 def gender_bias_score(word):  
2     return model.similarity(word, 'man') - model.similarity(word,  
3         'woman')  
4 # Results from Google News Word2Vec:  
5 # programmer: +0.091 (toward man)  
6 # nurse: -0.109 (toward woman)
```

Why this matters

Embeddings used in hiring tools, search, and recommendations can perpetuate bias!

Summary

What we learned

1. **Word2Vec (2013)**: neural revolution — predict context from words
 - Skip-gram most common, negative sampling for efficiency
2. **GloVe (2014)**: global co-occurrence statistics
3. **FastText (2017)**: character n-grams handle OOV
4. **Bias**: embeddings reflect training data biases

An important limitation to think about...

Word embeddings are fundamentally **static**: each word has a single vector representation, regardless of context. This means they cannot capture the different meanings a word might have in different sentences (e.g., "bank" as a financial institution vs. "bank" as the side of a river). Contextual embeddings (like ELMo, BERT) address this limitation by generating word representations that depend on the surrounding words.

Assignment 3: Wikipedia Embeddings

What you'll do

- Implement **10+ embedding methods** (LSA, Word2Vec, GloVe, FastText, SBERT, BGE, E5, ...)
- Visualize with **UMAP, clustering, and DataMapPlot**
- Evaluate via **document matching** (can embeddings match first/second halves?)
- Write some **reflection essays** on trade-offs and meaning

Due: February 2, 11:59 PM EST

Assignment 3: Wikipedia Embeddings

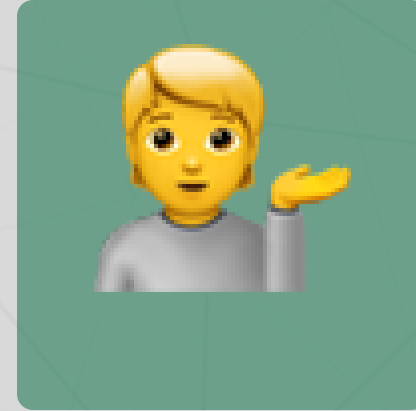
Questions?



Email me



Join our Discord



Come to office hours

Up next...

Contextual embeddings (ELMo, BERT)! These models construct embeddings based on the entire sentence, allowing for context-sensitive representations.